

Verteilte Systeme, Containerisierung

Vorwort:

Grundlage für diese Aufgabe ist die Präsentation «05_Messaging_RabbitMQ»

Dauer: 105 Minuten

Besprechung: 25 Minuten

Total: 130 Minuten

Ziel: Die Standalone-Simulation wird zu einem event-getriebenen System. RabbitMQ entkoppelt Producer und Consumer; Queue-Rückstau, Competing Consumers und Dead Letter Queue werden direkt sichtbar.

SETUP

Startpunkt ist Ihr Projekt nach Block 4. Falls dieser Stand fehlt, installieren Sie zuerst Foundation v1.0.0 und Block 4 v1.1.1. Für Block 5 benötigen Sie Zugriff auf das private Release v1.1.0. Docker Desktop, k3d, kubectl, Git und ein Browser genügen; make, jq und zusätzliche Programme sind nicht nötig. Die Befehle sind für PowerShell sowie Bash/WSL/Git Bash/macOS angegeben.

Material:

- Foundation: <https://github.com/SwitzerChees/vsc-dispatch-city-03-foundation/releases/tag/v1.0.0>
- Block 4: <https://github.com/SwitzerChees/vsc-dispatch-city-04-ingress/releases/tag/v1.1.1>
- Block 5: <https://github.com/SwitzerChees/vsc-dispatch-city-05-messaging/releases/tag/v1.1.0>

Aufgabe 1)

Messaging-Baustein integrieren und Topologie rendern

Übernehmen Sie das Block-5-Paket in Ihren Projektstand aus Block 4 und prüfen Sie die geplanten Ressourcen, bevor Sie den Cluster verändern.

- Klonen Sie das private Release einmalig neben Ihr Projekt-Repository.
- Führen Sie das Installationsskript im Wurzelverzeichnis Ihres Projekts aus.
- Rendern Sie das Overlay und suchen Sie Betriebsmodus, Broker-Image und Worker.

PowerShell

```
git clone --branch v1.1.0 --depth 1 https://github.com/SwitzerChees/vsc-dispatch-city-05-messaging.git ..\vsc-dispatch-city-05-messaging
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
& "..\vsc-dispatch-city-05-messaging\install.ps1" -Target "."
kubectl kustomize ./deploy/overlays/block-05-messaging
```

Bash / WSL / Git Bash / macOS

```
git clone --branch v1.1.0 --depth 1 https://github.com/SwitzerChees/vsc-dispatch-city-05-messaging.git ..\vsc-dispatch-city-05-messaging
../vsc-dispatch-city-05-messaging/install.sh .
kubectl kustomize ./deploy/overlays/block-05-messaging
```

ERWARTETES RESULTAT

APP_MODE ist «distributed». Das Rendering enthält RabbitMQ 4.3.5, ein PVC sowie Customer-, Restaurant-, Courier- und Order-Worker.

Nachweis: Notieren Sie Betriebsmodus, RabbitMQ-Image und Anzahl der fachlichen Worker-Workloads. Release neben mein Projekt geklont und das Install-Skript im Projekt-Root ausgeführt:

```
git clone --branch v1.1.0 --depth 1 https://github.com/SwitzerChees/vsc-dispatch-city-05-messaging.git ..\vsc-dispatch-city-05-messaging
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
& "..\vsc-dispatch-city-05-messaging\install.ps1" -Target "."
kubectl kustomize ./deploy/overlays/block-05-messaging
```

Im gerenderten Output: simulation-config hat APP_MODE: distributed, TICK_MS: "2000". RabbitMQ läuft als StatefulSet mit Image rabbitmq:4.3.5-management-alpine und einem 1Gi-PVC (volumeClaimTemplate "data", ReadWriteOnce). Vier fachliche Worker-Kategorien: customer-simulator und courier-simulator (StatefulSets), order-worker (Deployment, 2 Replicas), restaurant-bowl/curry/pizza (je Deployment, 1 Replica) - macht 6 Workloads total.

```
git add . ; git commit -m "Block 5: Messaging-Baustein integriert" ; git push
```

Aufgabe 2)

Images bauen, importieren und den verteilten Stand starten

Bauen Sie alle bereitgestellten Services, importieren Sie die Images in «teko-k8s» und deployen Sie den vollständigen Block-5-Stand.

- Bauen und importieren Sie die fünf Go-Services sowie das Dashboard.
- Wenden Sie das Overlay an und warten Sie auf RabbitMQ und alle Workloads.
- Prüfen Sie Betriebsmodus, PVC und den Ready-Zustand.

PowerShell

```
./scripts/build-images.ps1
./scripts/load-images.ps1 -Cluster teko-k8s
```

Bash / WSL / Git Bash / macOS

```
./scripts/build-images.sh
CLUSTER=teko-k8s ./scripts/load-images.sh
```

Alle Systeme

```
kubectl config use-context k3d-teko-k8s
kubectl apply -k ./deploy/overlays/block-05-messaging
kubectl -n food-delivery rollout status statefulset/rabbitmq --timeout=300s
kubectl -n food-delivery wait --for=condition=Available deployment --all --timeout=300s
kubectl -n food-delivery wait --for=condition=Ready pod --all --timeout=300s
kubectl -n food-delivery get deploy,statefulset,pods,pvc
```

ERWARTETES RESULTAT

Alle Deployments und StatefulSets sind bereit. «data-rabbitmq-0» ist Bound und die ConfigMap meldet APP_MODE=distributed.

Nachweis: Notieren Sie Ready-Zustand von RabbitMQ, Dashboard, Order Worker und den PVC-Status.

Erst Images gebaut und geladen, dann Overlay angewendet:

```
./scripts/build-images.ps1
./scripts/load-images.ps1 -Cluster teko-k8s
kubectl config use-context k3d-teko-k8s
kubectl apply -k ./deploy/overlays/block-05-messaging
```

Beim Warten auf RabbitMQ ist es schiefgelaufen - Rollout wurde nie fertig, Pod blieb im CrashLoopBackOff:

```
kubectl -n food-delivery rollout status statefulset/rabbitmq --timeout=300s
```

Troubleshooting - describe und Logs angeschaut:

```
kubectl -n food-delivery describe pod rabbitmq-0
kubectl -n food-delivery logs rabbitmq-0 --tail=50
```

In den Events stand mehrfach "Liveness probe failed: command timed out (rabbitmq-diagnostics -q ping)". Nicht RabbitMQ selbst kaputt, sondern der Diagnostics-Befehl war unter der Last auf meinem Laptop zu langsam und wurde deshalb ständig gekillt. Erstmal Last rausgenommen:

```
kubectl -n food-delivery scale deployment restaurant-bowl restaurant-curry restaurant-pizza --replicas=0
kubectl -n food-delivery delete pod rabbitmq-0
```

Das war nur ein Pflaster, also dauerhaft im Manifest gefixt - Probes von exec auf tcpSocket umgestellt (deploy/overlays/block-05-messaging/rabbitmq.yaml):

```
kubectl apply -k ./deploy/overlays/block-05-messaging
```

Danach wieder hochskaliert und control-api neu gestartet:

```
kubectl -n food-delivery scale deployment restaurant-bowl restaurant-curry restaurant-pizza --replicas=1
kubectl -n food-delivery scale deployment order-worker --replicas=2
kubectl -n food-delivery delete pod -l app.kubernetes.io/name=control-api
kubectl -n food-delivery get deploy,statefulset,pods,pvc
```

deployment.apps/control-api	1/1	1	1
deployment.apps/dashboard	2/2	2	2
deployment.apps/order-worker	2/2	2	2
deployment.apps/restaurant-bowl	1/1	1	1
deployment.apps/restaurant-curry	1/1	1	1
deployment.apps/restaurant-pizza	1/1	1	1
statefulset.apps/rabbitmq	1/1		
statefulset.apps/courier-simulator	1/1		
statefulset.apps/customer-simulator	1/1		
persistentvolumeclaim/data-rabbitmq-0	Bound	1Gi	RWO local-path

Fazit: Crash lag an zu strengen Health-Checks für einen lokal laufenden Cluster, nicht an RabbitMQ selbst. Mit tcpSocket-Probes ist das jetzt dauerhaft im Manifest gelöst.

Aufgabe 3)

Eventfluss im Dashboard und in RabbitMQ verfolgen

Öffnen Sie Dispatch City und RabbitMQ Management parallel. Erzeugen Sie eine Bestellung und verfolgen Sie dieselbe fachliche Aktion durch Broker, Queue und Worker.

- Starten Sie je einen Port-Forward für Traefik und RabbitMQ Management.
- Prüfen Sie Exchange und Queues und erzeugen Sie im Dashboard eine Bestellung.
- Vergleichen Sie den Event Stream mit den Logs der Producer und Consumer.

```
Terminal 1 - Traefik
kubect1 -n kube-system port-forward service/traefik 8080:80
Terminal 2 - RabbitMQ
kubect1 -n food-delivery port-forward service/rabbitmq 15672:15672
Browser und Logs
http://localhost:8080/
http://localhost:15672/   Login: delivery / delivery
kubect1 -n food-delivery logs statefulset/customer-simulator --since=2m
kubect1 -n food-delivery logs deployment/restaurant-pizza --since=2m
kubect1 -n food-delivery logs statefulset/courier-simulator --since=2m
```

ERWARTETES RESULTAT

Dispatch City zeigt «DISTRIBUTED» und laufende Events. RabbitMQ zeigt den Topic-Exchange «food.events» und acht Queues.

Nachweis: Notieren Sie eine beobachtete Eventkette vom Producer bis zur Lieferung.

Zwei Port-Forwards parallel:

```
# Terminal 1: kubect1 -n kube-system port-forward service/traefik 8080:80
# Terminal 2: kubect1 -n food-delivery port-forward service/rabbitmq 15672:15672
```

Dashboard zeigt DISTRIBUTED, RabbitMQ (delivery/delivery) zeigt Exchange food.events. Bestellung ausgelöst, Order-ID durch die Logs verfolgt:

```
kubect1 -n food-delivery logs statefulset/customer-simulator --since=5m
kubect1 -n food-delivery logs deployment/restaurant-pizza --since=5m
kubect1 -n food-delivery logs statefulset/courier-simulator --since=5m
```

Order-ID aa01cc95-10c3-45f8-80a7-b8be37a99391 taucht in allen drei Logs auf, mit steigenden Zeitstempeln:

18:10:02.453	order published (restaurant-pizza)	customer-simulator
18:10:03.564	order processed -> order.accepted	restaurant-pizza
18:14:05.392	order delivered	courier-simulator

Producer, Restaurant-Worker und Courier sind nicht direkt verbunden, kommunizieren nur über RabbitMQ - trotzdem taucht dieselbe Order-ID chronologisch überall auf. Genau das belegt, dass Exchange/Queues wirklich funktionieren.

Sichtbare Queues: food.dead, live.control-api-<pod>, courier-dispatch, order-projection, restaurant.restaurant-pizza/-curry/-bowl.

Aufgabe 4)

Rückstau erzeugen und mit Competing Consumers abbauen

Stoppen Sie den Pizza-Consumer, publizieren Sie acht reproduzierbare Events und beobachten Sie die Queue. Zwei Worker-Pods bauen den Rückstau anschließend gemeinsam ab.

- Lassen Sie den RabbitMQ-Port-Forward laufen, skalieren Sie «restaurant-pizza» auf 0 und publizieren Sie acht Pizza-Events.
- Prüfen Sie `messages_ready`; skalieren Sie danach auf zwei Consumer.
- Vergleichen Sie Queue-Zustand und Logs beider Pods.

PowerShell

```
kubectl -n food-delivery scale deployment/restaurant-pizza --replicas=0
kubectl -n food-delivery wait --for=delete pod -l app.kubernetes.io/instance=restaurant-pizza --timeout=120s
./scripts/lab-publish.ps1 -Mode valid -Count 8
```

Bash / WSL / Git Bash / macOS

```
kubectl -n food-delivery scale deployment/restaurant-pizza --replicas=0
kubectl -n food-delivery wait --for=delete pod -l app.kubernetes.io/instance=restaurant-pizza --timeout=120s
./scripts/lab-publish.sh valid 8
```

Alle Systeme

```
kubectl -n food-delivery exec rabbitmq-0 -- rabbitmqctl list_queues name consumers messages_ready
kubectl -n food-delivery scale deployment/restaurant-pizza --replicas=2
kubectl -n food-delivery rollout status deployment/restaurant-pizza --timeout=180s
kubectl -n food-delivery exec rabbitmq-0 -- rabbitmqctl list_queues name consumers messages_ready
kubectl -n food-delivery logs -l app.kubernetes.io/instance=restaurant-pizza --prefix --since=2m
```

ERWARTETES RESULTAT

Ohne Consumer stehen acht Nachrichten auf «ready». Mit zwei Consumer-Pods fällt der Rückstau auf 0; beide Pod-Namen erscheinen in den Logs.

Nachweis: Halten Sie Queue-Werte vor/nach dem Skalieren und beide Pod-Namen fest.

Pizza-Consumer auf 0, gewartet bis Pod weg ist, dann 8 Events publiziert:

```
kubectl -n food-delivery scale deployment/restaurant-pizza --replicas=0
kubectl -n food-delivery wait --for=delete pod -l app.kubernetes.io/instance=restaurant-pizza --timeout=120s
./scripts/lab-publish.ps1 -Mode valid -Count 8
kubectl -n food-delivery exec rabbitmq-0 -- rabbitmqctl list_queues name consumers messages_ready
restaurant.restaurant-pizza consumers=0 messages_ready=8
```

Wie erwartet: 8 Nachrichten hängen fest, kein Consumer. Hochskaliert auf 2:

```
kubectl -n food-delivery scale deployment/restaurant-pizza --replicas=2
kubectl -n food-delivery rollout status deployment/restaurant-pizza --timeout=180s
kubectl -n food-delivery exec rabbitmq-0 -- rabbitmqctl list_queues name consumers messages_ready
restaurant.restaurant-pizza consumers=2 messages_ready=0
kubectl -n food-delivery logs -l app.kubernetes.io/instance=restaurant-pizza --prefix --since=2m
Pod restaurant-pizza-...-swbvq verarbeitete: lab-pizza-...-1, -3, -5, -7
Pod restaurant-pizza-...-ljcsr verarbeitete: lab-pizza-...-2, -4, -6, -8
```

Alle 8 mit `result=order.accepted`, nach ca. 3 Sekunden komplett durch. RabbitMQ hat die Nachrichten reihum auf beide aktiven Consumer verteilt (Competing Consumers).

Aufgabe 5)

Ungültige Nachricht in der Dead Letter Queue isolieren

Publizieren Sie absichtlich ungültiges JSON. Die Consumer bestätigen es nicht; RabbitMQ verschiebt die betroffenen Kopien in «food.dead».

- Lassen Sie den RabbitMQ-Port-Forward laufen und leeren Sie die DLQ für einen eindeutigen Ausgangswert.
- Publizieren Sie die ungültige Nachricht mit dem vorbereiteten Skript.
- Prüfen Sie die DLQ und stellen Sie den Pizza-Worker auf eine Replik zurück.

Vorbereiten – alle Systeme

```
kubectl -n food-delivery exec rabbitmq-0 -- rabbitmqctl purge_queue food.dead
```

PowerShell

```
./scripts/lab-publish.ps1 -Mode invalid
```

Bash / WSL / Git Bash / macOS

```
./scripts/lab-publish.sh invalid
```

Prüfen und Abschluss

```
kubectl -n food-delivery exec rabbitmq-0 -- rabbitmqctl list_queues name messages_ready
```

```
kubectl -n food-delivery scale deployment/restaurant-pizza --replicas=1
```

```
kubectl -n food-delivery rollout status deployment/restaurant-pizza --timeout=180s
```

ERWARTETES RESULTAT

«food.dead» enthält mindestens zwei Nachrichtenkopien. Die normalen Queues bleiben verarbeitetbar und der Pizza-Worker endet wieder bei 1/1.

Nachweis: Notieren Sie den DLQ-Wert und erklären Sie kurz, warum dieselbe Publikation mehrfach dead-lettered wird. DLQ vorher geleert für einen sauberen Ausgangswert, dann ungültiges JSON publiziert:

```
kubectl -n food-delivery exec rabbitmq-0 -- rabbitmqctl purge_queue food.dead
./scripts/lab-publish.ps1 -Mode invalid
kubectl -n food-delivery exec rabbitmq-0 -- rabbitmqctl list_queues name messages_ready
food.dead                messages_ready = 2
live.control-api-5d947b8f7-9lwc4  messages_ready = 0
courier-dispatch          messages_ready = 33
order-projection          messages_ready = 17
restaurant.restaurant-pizza/-curry/-bowl  messages_ready = 0
```

Zwei Kopien in food.dead (wie gefordert, mind. 2), alle anderen Queues bleiben unberührt. Grund: RabbitMQ liefert eine nicht bestätigte Nachricht (Consumer lehnt ungültiges JSON ab) erneut zu, jeder abgelehnte Zustellversuch wird separat über die Dead-Letter-Exchange nach food.dead geschrieben - deshalb mehrere Kopien derselben Publikation.

Pizza-Worker wieder auf 1 zurückgestellt:

```
kubectl -n food-delivery scale deployment/restaurant-pizza --replicas=1
kubectl -n food-delivery rollout status deployment/restaurant-pizza --timeout=180s
deployment "restaurant-pizza" successfully rolled out (1/1)
```